EDE PLATFORM · ENGINEERING EXPLAINER

Hook-Driven Validation in Business Models

How EDE enforces business rules and constraints on domain models — the lifecycle-hook mechanism, where it fires, and how to author one correctly.

> **Verdict / TL;DR:** Yes — hook-driven validation is EDE's canonical constraint mechanism. Every mutating command routes through the CommandBus, which fires synchronous **pre.{model}.{op}** hooks; a hook raises an exception to **veto** the whole operation inside its transaction. Field-level declarations and validate() cover shape; hooks cover business rules.

PREPARED FOR
EDE developers

SOURCE INPUTS
core/bus/command_bus.py · core/env.py · core/api.py

DATE
10 July 2026

SECTION 01

01 Where validation lives — the three layers

EDE has no `@api.constrains` decorator. Constraints are enforced across three layers, cheapest to most powerful. Each answers a different question, and you combine them.

3	pre · post	raise	4
CONSTRAINT LAYERS	HOOK PHASES PER OP	= VETO OPERATION	READ CMDS EXCLUDED

LAYER 1 — Field-level

Is the shape valid — required, unique, length, FK?

`required=True` · `unique=True` · `Char(max_length, regex)` · `Reference(on_delete)`

LAYER 2 — Lifecycle hooks

Is this business rule satisfied — conditional, cross-field, cross-record?

`@api.on_hook("pre.crm.lead.create")` → `raise ValueError` (veto)

LAYER 3 — validate()

A simple self-contained invariant, checked on save().

`def validate(self)` → `raise on bad state`

Which layer for which job

Layer	Use it for	Example	Role
Field-level	Shape: presence, uniqueness, string length/pattern, referential behaviour	`code = fields.Char(required=True, unique=True)`	DECLARE
Lifecycle hook	Business rules: conditional checks, cross-field logic, lookups against other records	"can't mark *won* without a partner"	CANONICAL
validate()	Basic self-contained invariants on a single record's own fields	`min_qty <= max_qty`	SIMPLE

Hooks are the workhorse

Field declarations cover the "shape" of data and the database enforces it. But real ERP **business rules** — the ones that depend on state, other fields, or other records — live in **pre-hooks**. That is what "hook-driven validation" means, and it is where most domain constraints belong.

SECTION 02

02 How a validation hook fires

Every **mutating** ORM operation is a `Command` dispatched through the `CommandBus`. The bus derives a hook key from the command, fires **pre-hooks**, runs the handler, then fires **post-hooks**. Pre-hooks that raise abort the command before it commits.

STEP 1 · DISPATCH
CommandBus
The ORM turns create/write/delete into a Command.

`proxy.create(...)` → `Command` ede.create → `CommandBus.dispatch`

STEP 2 · PRE-HOOKS
pre.{model}.{op}
Validation runs here. Raise to veto — nothing commits.

`env.run_hooks("pre.crm.lead.create")` → `raise ⇒ abort`

STEP 3 · HANDLER
CRUD / domain handler
Only reached if every pre-hook passed.

`write to DB` → `emit ede.record.created`

STEP 4 · POST-HOOKS
post.{model}.{op}
Runs after success — cache busting, side effects, audit.

`env.run_hooks("post.crm.lead.create")`

How the hook key is derived

Command + model	Hook key fired	Kind
`ede.create` on `crm.lead`	`pre./post.crm.lead.create`	Generic CRUD
`ede.update` on `crm.lead`	`pre./post.crm.lead.update`	Generic CRUD
`ede.delete` on `crm.lead`	`pre./post.crm.lead.delete`	Generic CRUD
`crm.lead.qualify` (domain cmd)	`pre./post.crm.lead.qualify`	Domain command
any model, `delete`	`pre.*.delete` (wildcard fan-out)	Cross-cutting

> ⚠ **Reads never fire hooks**
> The four read-only commands — `ede.search`, `ede.read_one`, `ede.count`, `ede.read_group` — are excluded. Validation only runs on **mutations**. Don't try to "guard a read" with a hook; it won't fire.

`cmd.record` — what you inspect inside the hook

Phase	What `cmd.record` holds
pre-create	A `__new__` in-memory RecordSet built from the incoming values (no DB row yet)
pre-update / pre-delete	The **live DB record** as it exists *before* the change
incoming values	Always at `(cmd.payload or {}).get("values")` — merge with the record to validate the resulting state

SECTION 03

03 Writing a validation hook — patterns

Declare hooks as methods on the `DomainModel`. They are auto-scanned at registration; `self.env` is injected, so you can search other models from inside. Raise any exception to veto.

PATTERN A — CREATE: VALIDATE THE INCOMING VALUES

```
@api.on_hook("pre.crm.lead.create")
def _check_on_create(self, cmd: Command) -> None:
    values = (cmd.payload or {}).get("values") or {}
    if values.get("expected_revenue", 0) < 0:
        raise ValueError("expected_revenue cannot be negative")
```

PATTERN B — UPDATE: MERGE LIVE RECORD + INCOMING TO CHECK THE RESULT STATE

```
@api.on_hook("pre.crm.lead.update")
def _check_on_update(self, cmd: Command) -> None:
    record = cmd.record                              # live DB record
    values = (cmd.payload or {}).get("values") or {}
    new_stage = values.get("stage", record.stage)   # fall back to current
    if new_stage == "won" and not record.partner_id:
        raise ValueError("a lead cannot be marked won without a partner")
```

PATTERN C — CROSS-RECORD: USE ENV TO LOOK UP OTHER MODELS (CONDITIONAL UNIQUENESS, REFS)

```
@api.on_hook("pre.crm.lead.create")
def _unique_active_code(self, cmd: Command) -> None:
    code = (cmd.payload or {}).get("values", {}).get("code")
    # reads go direct through the model proxy — never through the bus
    clash = self.env.models["crm.lead"].search(
        [("code", "=", code), ("active", "=", True)])
    if clash:
        raise ValueError(f"code {code} already in use by an active lead")
```

PATTERN D — DELETE GUARD (AND CROSS-CUTTING WILDCARD)

```
@api.on_hook("pre.crm.lead.delete")
def _block_won_delete(self, cmd: Command) -> None:
    if cmd.record.stage == "won":
        raise ValueError("won leads are retained for audit — archive instead")
# A free-function hook on "pre.*.delete" fires for EVERY model's delete.
```

The whole command is transactional

Because the pre-hook runs before the handler and inside the command's transaction, raising leaves the database untouched — no partial write, no cleanup needed. The exception propagates to the caller unchanged.

SECTION 04

04 Rules, gotchas & best practice

A few conventions keep hook validation predictable and keep it from turning into a performance or architecture problem.

Do / Don't

Guidance	Why	
Read other records with `env.models["k"].search(...)` inside the hook	Reads must go direct through the proxy — the bus is for mutations only	DO
On update, merge `cmd.record` + payload values before checking	Payload may be partial; validate the *resulting* state, not just the delta	DO
Raise a clear, message-bearing exception	The message surfaces to the API caller / UI toast unchanged	DO
Put the check in a `post` -hook when you meant to block	By then the write already happened; only `pre` can veto	DON'T
Dispatch a read `Command("ede.search")` from the hook	Adds dispatcher + middleware overhead for zero gain; hooks don't fire on reads anyway	DON'T
Expect a hook to run on `search` / `read_one` / `count` / `read_group`	Read-only commands are excluded by design	DON'T

NAMING & PLACEMENT

- Key pattern is `pre.{model_key}.{op}` / `post.{model_key}.{op}`.
- Ops are `create` · `update` · `delete` plus any domain command name.
- Model-method hooks live on the `DomainModel`; cross-cutting ones can be free functions on `pre.*.{op}`.

FIELD-LEVEL VS HOOK — PICK RIGHT

- Presence / uniqueness / length / regex / FK → declare on the **field**.
- Uniqueness enforced via a **named index**; change the field then `ede migrate generate`.
- Anything conditional or cross-record → a **pre-hook**.

⚠ Hooks are framework-shape

Adding, removing, or reordering a hook on a platform (`res.*` / `ir.*`) model changes kernel behaviour. On the EDE codebase, that class of change requires explicit approval before editing under `src/ede/core/` or a platform field declaration — don't add a hook just to silence a failing test.

Bottom line

For a business-model constraint: reach for a **field-level** declaration for shape, and a `@api.on_hook("pre.{model}.{op}")` that **raises to veto** for the business rule. That pre-hook, running transactionally before the write, is EDE's hook-driven validation.